



Agentic AI & Multiagent Systems

Working with LangChain & LangGraph



Semester Course of 03 Credits

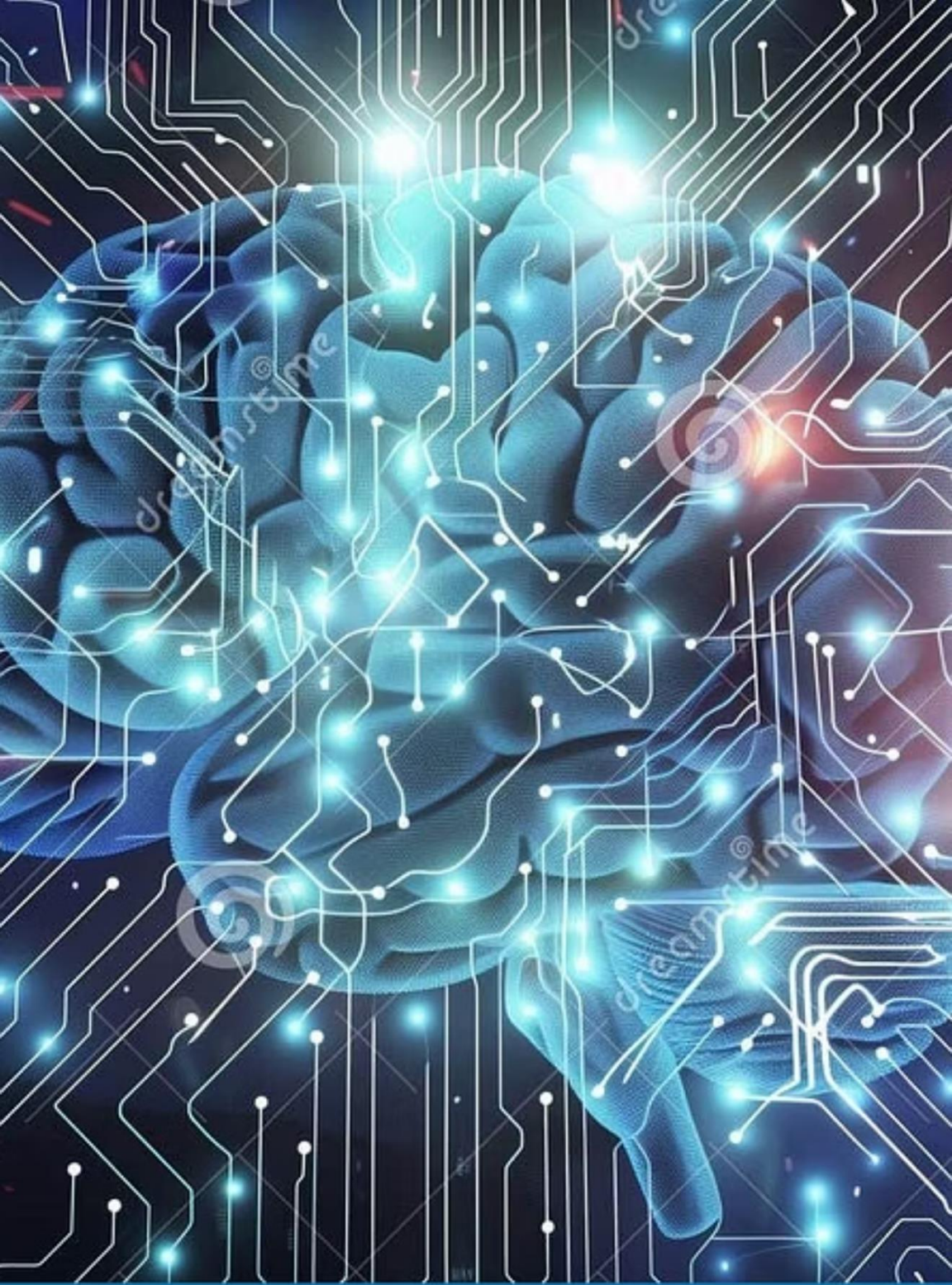
B.Tech. 6th Sem / M.Tech. 2nd Sem

Prof Giovanni Merlino
Associate Professor (Computer Engineering)
University Of Messina, Italy

Prof Dr. Om Prakash Vyas
Vice Chancellor & Director
IIIT Naya Raipur, India

Outline

- ❖ Introduction to LangChain and LangGraph
- ❖ The Core Agent Loop: Revisiting the Basics
 - ❖ **ReAct**
 - ❖ **Key concepts of AI Agent**
 - ❖ **The Agent Reasoning Loop**
- ❖ Bridging between LLM & Real Applications:
 - ❖ **LLM is Agent ?**
 - ❖ API, Database, Tools, Memory & External Knowledge
 - ❖ **The Four Pillars of LangChain Agents**
 - ❖ **What LangGraph does?**
 - ❖ **LangChain vs. LangGraph: Knowing the connection**
- ❖ What they do together: LangChain and LangGraph
- ❖ Tech Stack: Core Frameworks & Supporting Tools
- ❖ Common Mistakes to be avoided
 - ❖ **Overuse of LLMs, Vague Tool Description, Weak Prompt Design**
- ❖ Foundational Papers to study...
- ❖ Start Small- Build progressively..
 - ❖ **From Prompt Engineering to Agents and then MultiAgent Coordination**



Agentic AI with LangChain & LangGraph

What's the role of **LangChain & LangGraph** in practical, hands-on for the course
Agentic AI
How can it help students build autonomous AI agents that **reason, plan, and act** using
real tools.

In a course on Agentic AI, frameworks like LangChain play a central enabling role because they **provide the infrastructure to convert a Large Language Model into an AI agent** capable of **reasoning, planning, using tools**, and **interacting** with external systems.

For a hands-on course, LangChain helps us move
from “**prompting LLMs**” to “building **autonomous AI systems**.”

LangChain: What & How

LangChain bridges the **gap between raw language models and fully functional, real-world AI agents** — covering integration components, reasoning loops, and the architecture that makes agentic behaviour possible.

The Core Objective

Our objective is to have design, build, and deploy **autonomous AI agents** that can reason through complex problems, select the right tools, and take sequential actions to achieve goals — without step-by-step human instruction.

Why LangChain?

These two **frameworks** offer the most practical on-ramp to agentic AI available today.

LangChain provides rapid prototyping of tool-using assistants.



We learn by **building**, not just **reading** about AI.

The **Simplest** Definition to Give....

"LangChain helps us convert a language model into a **useful intelligent agent** by connecting it with tools, memory, and data."

If you leave this course remembering only one sentence, make it this one.
It captures the core value proposition – that an LLM alone is a starting point, not a finished product – and it frames every lab and exercise as a step towards that goal.

Tools

Connect to the world – APIs, search, files, calculators

Memory

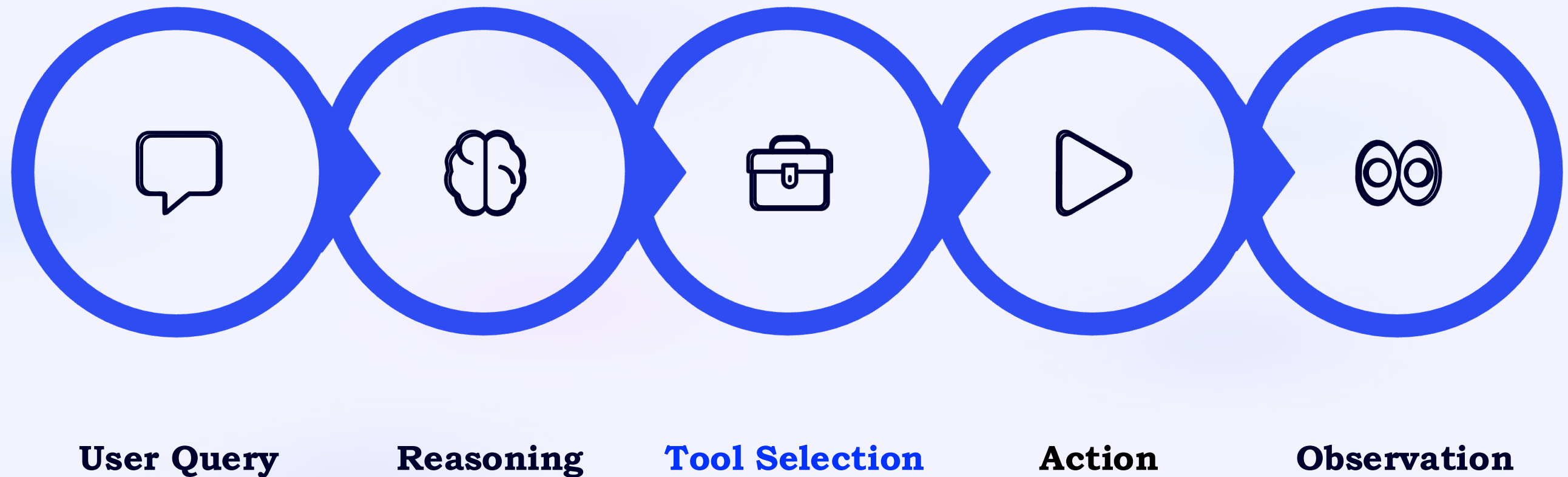
Retain context across turns and sessions

Data

Ground responses in real, up-to-date information

The Core Agent Loop: ReAct (Reason + Act)

The foundation of agentic AI is the **ReAct (Reason + Act)** loop — a cycle where the agent continuously reasons about its goal, selects an action, observes the result, and iterates until the task is complete.



This loop is the engine behind virtually every modern AI agent. Understanding it deeply is a prerequisite for everything that follows.

Key Concepts: AI Agents



Agents

Autonomous systems that perceive their environment and take goal-directed actions using an LLM as the reasoning core.



Memory

Short-term and long-term storage that allows agents to retain context across steps, sessions, and multi-turn interactions.



Environment

The broader context in which the agent operates – including APIs, databases, user input, and the web.



Tools

External capabilities the agent can invoke – APIs, calculators, search engines, file readers – to gather information or take actions.



Planning

The ability to decompose a complex goal into sub-tasks and determine the optimal sequence of actions to achieve it.



Feedback Loops

Mechanisms by which agents observe outcomes, update their reasoning, and refine subsequent actions dynamically.

LangChain: Bridge Between LLMs and Real Applications

A large language model on its own — whether **GPT-4**, **Llama**, or **Claude** — is a powerful text predictor.

It becomes a **genuine agent** only when it is connected to the outside world. Agentic AI requires seamless integration with:

APIs

Call external services and retrieve live data in real time.

Databases

Read and write structured or vector-based data stores.

Tools

Execute code, search the web, or run custom functions.

Memory

Retain context across turns and longer sessions.

External Knowledge

Retrieve domain-specific documents via RAG pipelines.

LangChain provides **ready-made components** for every one of these integration layers, dramatically reducing the engineering overhead of building production-grade agents.

LLM is Agent ?

"LLMs alone are not agents." They become agents when integrated with tools, memory, and reasoning loops.

This is one of the most important conceptual shifts we need to make when moving from prompt engineering to agentic AI.

The distinction shapes every architectural decision that follows.

LLM Alone

- Stateless text generation
- No persistent memory
- Cannot take actions
- Single-turn reasoning only

LLM + LangChain Integration

- Connected to tools and APIs
- Short- and long-term memory
- Can execute multi-step tasks
- Iterative reasoning loops



LangChain Introduction

LangChain is a **framework for building applications powered by Large Language Models (LLMs)**. Think of it as a toolkit that connects an AI model to external tools, data sources, and memory – transforming a bare LLM into a capable, context-aware agent.

Without LangChain

User → LLM → Answer

The model can only respond based on what it was trained on. No tools, no memory, no external data.

With LangChain

**User → LLM → Tools → Data →
Memory → Answer**

The AI can search the web, read documents, query databases, run calculations, and remember prior context.

LangChain vs. LangGraph: Knowing the connection

LangGraph is built **on top of LangChain** and represents agents as **state graphs** — a powerful abstraction that enables complex, branching, looping agent workflows.

LangChain

Build LLM apps

Rapid prototyping of chains, retrievers, and simple tool-using agents.
Great for linear workflows and getting started fast.

LangGraph

Build autonomous agents

Full state machine control. Supports multi-agent workflows, persistent memory, planning, conditional branching, and execution loops.

Key LangGraph capabilities: **multi-agent workflows**, **memory**, **planning**, **loops**, and **persistent state** across sessions — the building blocks of truly autonomous systems.

Building AI Agents with LangChain: Role of LangGraph

The centrepiece of agentic AI is the ability to **decide what action to take next** based on the current context, available tools, and a stated goal. LangChain's Agent framework is purpose-built for exactly this.

The most important concept in Agentic AI is agents that can **decide actions dynamically** — not just generate text, but reason about what to do and then do it.

Limitation of LangChain for Agents

LangChain alone works well for **simple agents**, but **complex agents require structured control flow**:

- multi-step reasoning
- loops
- decision branches
- multi-agent collaboration
- long workflows

This is where **LangGraph** becomes important.

While **LangGraph** enables true multi-agent architectures with persistent state and planning loops.

Role of LangGraph in Agentic AI

LangGraph is used to **design the control flow of agents**.

It models an AI agent as a **graph of states and actions**.

Concept

Agent workflow = **Graph**

Nodes = operations

Edges = transitions

This allows:

looping

conditional branching

memory persistence

multi-agent systems

The Four Pillars of LangChain Agents

LangChain's [Agent framework](#) enables four core capabilities that together produce genuinely autonomous behaviour:



Tool Selection

The agent inspects available tools – search, calculator, code executor, database – and selects the most appropriate one for each sub-task.



Reasoning Loops

Using frameworks like ReAct, the agent alternates between thinking ("what do I know?") and acting ("what should I do next?") in iterative cycles.



Planning

Before acting, the agent decomposes a high-level goal into a sequence of manageable sub-goals, producing a structured plan.

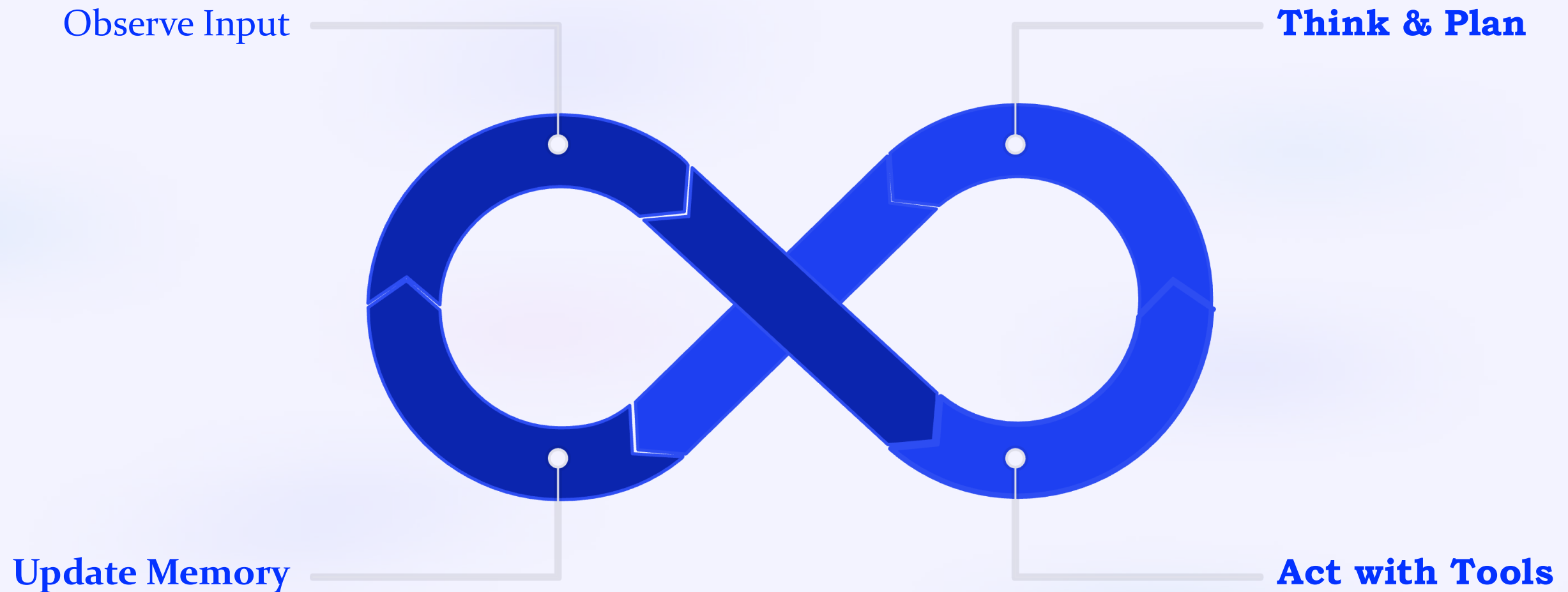


Execution

The agent carries out each planned step – calling tools, processing results, updating its state – until the goal is achieved.

LangChain & the Agent Reasoning Loop

At runtime, a LangChain agent operates in a continuous **Observe** → **Think** → **Act** cycle. This loop continues until the agent determines it has satisfied the user's goal or exhausts its step budget.



Understanding this loop is foundational — every agentic pattern students will build, from simple chatbots to multi-agent pipelines, is a variation on this cycle.

Core Building Blocks of LangChain

Students should internalise **five fundamental components** before writing a single line of agent code. Use this analogy consistently throughout your course to build intuition fast.



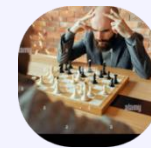
LLM — The Brain



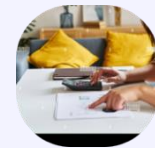
Prompt: The Instructions



Chains: The Pipeline



Agent: The Decision Maker



Tools — The Hands



Core Building Blocks of LangChain

Students should internalise **five fundamental components** before writing a single line of agent code. Use this analogy consistently throughout your course to build intuition fast.



LLM — The Brain

The underlying language model (GPT-4, Llama 3, Mistral, etc.) that understands language and generates responses.



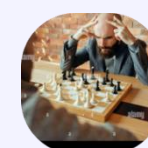
Prompt: The Instructions

Structured text that tells the LLM how to behave, what role to play, and what format to respond in.



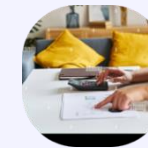
Chains: The Pipeline

A sequence of operations linking prompts, LLMs, tools, and outputs into a reproducible workflow.



Agent: The Decision Maker

The reasoning layer that decides *which* tool to use, *when* to use it, and how to interpret the result.



Tools — The Hands

External capabilities the agent can invoke: web search, calculators, APIs, file readers, and more.

How This Leads to Agentic AI

When you combine:

- LLM
- Tools
- Memory
- Decision-making

you get an AI Agent.

This is where **LangGraph** becomes useful, because it helps manage **complex multi-step agent workflows**.

Minimum Tech Stack for Students

Core Frameworks



LangChain

Agent construction, tool integration, chains, and retrieval-augmented generation.



LangGraph

Stateful multi-agent workflows, planning loops, and persistent memory management.



Python

The primary language. Students should be comfortable with Python 3.10+ before the course begins.

Supporting Tools



Streamlit

Rapid UI layer – lets students build shareable web interfaces for their agents in minutes, without frontend experience.



FAISS

Vector store for Retrieval-Augmented Generation (RAG) – enables agents to query large document collections efficiently.



Ollama or OpenAI API

LLM backend. Ollama enables fully local models; OpenAI API provides GPT-4 access. Offer both to accommodate different setups.

Common Mistakes Students Make

Forewarned is forearmed. Share these pitfalls explicitly in Week 1 so students can avoid frustrating debugging sessions later. Most of these mistakes stem from treating the LLM as magic rather than as a system with well-defined inputs and outputs.

1

Overusing the LLM

Students send every sub-task to the LLM, including things better handled by deterministic code. Teach them to reserve the LLM for language and reasoning tasks only.

2

Vague Tool Descriptions

The agent decides which tool to use based on the `description` field. A poor description like *"does stuff"* causes the agent to pick the wrong tool – or no tool at all.

3

Weak Prompt Design

Unclear system prompts lead to inconsistent, unpredictable behaviour. Prompt engineering is a skill – dedicate Week 1 to it before anything else.

4

No Error Handling

Agents that crash silently are impossible to debug. Teach students to wrap tool calls in try/except blocks and log intermediate steps with `verbose=True`.

Lab Exercise 1

Build an AI Assistant with a Calculator

This first exercise is intentionally simple. The goal is not to build something impressive – it is to make the **agent reasoning process visible** to students for the first time. When they see the Thought → Action → Observation loop printed in the terminal, the concept of agentic AI clicks.

Installation

```
pip install langchain openai  
# or for local models:  
pip install langchain ollama
```

What Students Will See

```
Thought: I should use calculator  
Action: Calculator  
Observation: 9315  
Final Answer: 9315
```

The Code

```
from langchain.agents import initialize_agent  
from langchain.tools import Tool  
from langchain.llms import OpenAI  
  
llm = OpenAI()  
tools = [  
    Tool(  
        name="Calculator",  
        func=lambda x: eval(x),  
        description="Useful for math"  
    )  
]  
agent = initialize_agent(  
    tools, llm,  
    agent="zero-shot-react-description",  
    verbose=True  
)  
agent.run("What is 345 * 27?")
```

Lab Idea 1

Document Question-Answering Bot

Students upload a PDF – a research paper, a policy document, a textbook chapter – and build a bot that can answer natural-language questions about it. This lab introduces the full **Retrieval-Augmented Generation (RAG)** pipeline and is one of the most immediately useful skills in the course.

Convert to Embeddings

Generate dense vector representations.

Store in FAISS

Index embeddings in a FAISS vector database.



Upload PDF

Student uploads research or policy document.

Ask Questions

Query in natural language for answers.

Use **FAISS** or **Chroma** as the vector store. A practical prompt to give students: *"Ask three questions about this research paper that you could not answer without reading it."* This forces meaningful retrieval rather than trivial lookups.

Lab Idea 2

AI Research Assistant

Students build an agent that behaves like a programmable version of ChatGPT's browsing mode. Given a question, it searches the web, reads relevant sources, synthesises findings, and returns a structured report. Students consistently rate this as their favourite lab because the output feels genuinely impressive – and they built it themselves.

Agent Workflow

- User submits a research question
- Agent searches the web for relevant sources
- Agent reads and extracts key information
- Agent summarises and returns a structured report

Tools to Wire In

Wikipedia

Encyclopaedic background knowledge

ArXiv

Academic papers and preprints

Web Search

DuckDuckGo or SerpAPI for live results

Lab Idea 3

AI Data Analyst

Students give their agent a CSV file and a plain-English prompt. The agent reads the data, calculates relevant statistics, identifies patterns, and returns a human-readable summary. This lab bridges data science and NLP in a way that resonates strongly with students from both backgrounds.

Read Structured Data

The agent loads CSV files using pandas under the hood, making structured data accessible to the LLM without manual preprocessing.

Calculate Statistics

Students prompt the agent to compute means, totals, rankings, and correlations – all expressed in natural language rather than code.

Generate Insight Summaries

The agent returns narrative explanations of findings. Example prompt: *"Analyse this sales dataset and identify the top-performing region."*

Foundational Papers to Assign

ReAct

Introduces the Reason + Act paradigm.
Establishes how LLMs can interleave reasoning traces with tool-use actions to solve tasks more reliably.

Tree-of-Thoughts

Explores how LLMs can deliberate over multiple reasoning paths simultaneously, enabling backtracking and more systematic problem-solving.

AutoGPT Architecture

An early real-world demonstration of autonomous, self-prompting agents – a reference point for discussing both the promise and limitations of current agents.

First Lab: Build a Tool-Using AI Assistant

Tools Students Will Wire Up



Calculator

Perform precise arithmetic without relying on the LLM's unreliable mental maths.



Wikipedia Search

Retrieve factual information dynamically from a live knowledge source.



Weather API

Query real-time data – a tangible demonstration of agents interacting with the live web.



File Reader

Read and process local files, setting the stage for RAG pipelines in later weeks.

Example Code Structure

```
from langchain.agents import initialize_agent
from langchain.tools import Tool
from langchain.llms import OpenAI

tools = [
    Tool(
        name="Calculator",
        func=calculator.run
    ),
]

agent = initialize_agent(
    tools,
    llm,
    agent="zero-shot-react-description"
)

agent.run(
    "What is GDP of India
    divided by population?"
)
```

LangChain vs. LangGraph: Knowing the Difference

LangGraph is built **on top of LangChain** and represents agents as **state graphs** — a powerful abstraction that enables complex, branching, looping agent workflows.

LangChain

Build LLM apps

Rapid prototyping of chains, retrievers, and simple tool-using agents.
Great for linear workflows and getting started fast.

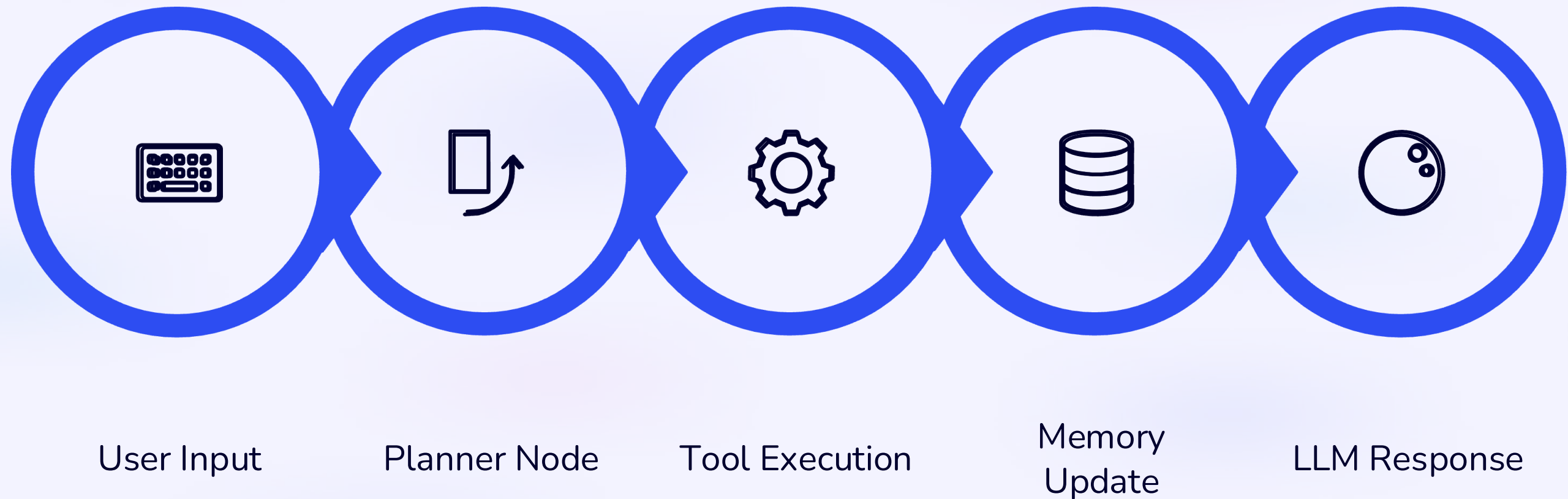
LangGraph

Build autonomous agents

Full state machine control. Supports multi-agent workflows, persistent memory, planning, conditional branching, and execution loops.

Key LangGraph capabilities: **multi-agent workflows**, **memory**, **planning**, **loops**, and **persistent state** across sessions — the building blocks of truly autonomous systems.

LangGraph Agent: State Graph Flow



In LangGraph, each node in the graph represents a discrete operation – planning, tool use, memory retrieval, or response generation. Edges define the transitions, including conditional loops that allow the agent to keep working until it satisfies the termination condition.

AI Research Assistant: Agent Workflow

What the Agent Does

01

Understand the Research Topic

Parses the user's query, identifies key concepts, and formulates targeted search terms.

02

Search for Papers

Queries the Arxiv API to retrieve relevant academic papers based on the refined search terms.

03

Read & Summarise Papers

Uses a PDF reader tool to extract content and a summariser to condense key findings.

04

Generate a Report

Synthesises summaries into a structured research report with citations and key takeaways.

Tools Required



Arxiv API

Programmatic access to millions of academic preprints in CS, physics, mathematics, and AI.



PDF Reader

Extracts and parses text from downloaded paper PDFs for processing by the LLM.



Summariser

A chain or LLM call that condenses lengthy academic text into structured, readable summaries.

Final Project Options



Autonomous Data Analyst

Ingests raw datasets, formulates hypotheses, runs analyses, and generates narrative insight reports without manual prompting.



AI Personal Tutor

Adapts to a student's knowledge level, generates practice problems, provides feedback, and tracks progress over time.



AI Startup Advisor

A multi-agent system that evaluates business ideas across market, technical, and financial dimensions, delivering actionable recommendations.



Autonomous Web Research Agent

Browses the web, aggregates information from multiple sources, cross-references facts, and compiles structured research briefs.

Agentic AI & Multiagent Systems

Key Takeaways.....

Build First

Prioritise hands-on labs over lectures. Students understand agent loops far better after seeing them execute live than from diagrams alone.

Scaffold Complexity

LangChain → LangGraph → Multi-Agent is the right progression. Each layer adds capability without overwhelming students before they're ready.

Real Projects Matter

Final projects should solve a problem students *actually care about*. Intrinsic motivation dramatically improves the quality of capstone work.

Minimal Stack

Resist tool proliferation. Python, LangChain, LangGraph, Streamlit, and FAISS is everything students need to build production-quality agents.